

Hermass AI Quant Platform — 最终版实施方案

> 版本：v3.0 Final > 日期：2026-06-06 > 基础模型：Qoder（架构 + DSL + Agent 体系 + 里程碑）> 融合来源：Kimi（性能 + 前沿研究 + 产业链）/ Codex（协议 + 可观测性 + Typed Contracts）/ DeepSeek（合规 + 测试 + 运维）>> 决策说明：经四模型交叉对比，Qoder 与现有 Hermass 架构（hermass_platform / agently_adapter / backtest / DuckDB 资产）兼容性最优，功能链路最完整，安全约束最严格（DSL 层不执行代码）。本文档以 Qoder 为骨架，融合 Kimi 的性能与前沿探索、Codex 的协议与可观测性、DeepSeek 的合规与测试规范，形成可执行、可验收、可演进的最终方案。

目录

[总体架构](#1-总体架构) 2. [产品功能规格](#2-产品功能规格) 3. [技术架构详细设计](#3-技术架构详细设计) 4. [数据库设计](#4-数据库设计) 5. [Agent 体系设计](#5-agent-体系设计) 6. [API 接口规范](#6-api-接口规范) 7. [安全、合规与风控](#7-安全合规与风控) 8. [可观测性设计](#8-可观测性设计) 9. [测试规范](#9-测试规范) 10. [实施路线图](#10-实施路线图) 11. [验收标准](#11-验收标准) 12. [风险矩阵与缓解](#12-风险矩阵与缓解) 13. [成本估算](#13-成本估算) 14. [附录](#14-附录)

1. 总体架构

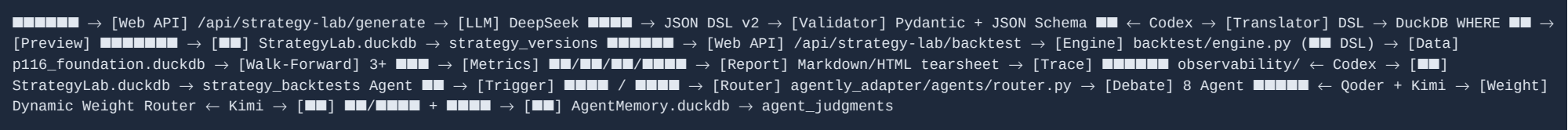
1.1 架构原则

Web 层只做入口：请求聚合、模板渲染、路由转发，不含业务逻辑。2. DSL 为策略唯一表达：用户策略必须先变成结构化 JSON，不可执行 LLM 生成的 Python。3. Agently 层负责编排：Agent 调用、DAG 执行、多步工作流程统一由 agently_adapter 管理。4. Hermes 记忆层持久化：AgentMemory.duckdb 是判断、复盘、进化的唯一真相源。5. 红线不可绕过：所有策略修改/执行路径必须经过 hermass_platform/red_lines.py。6. 可追溯、可回滚：每次策略变更、回测、执行都写入版本化的审计日志。7. 兼容层先行：MCP / Typed Contracts / Tracing 以 Adapter 方式接入，不推翻现有运行时。8. 前沿研究隔离：TS-FM / RAG-KG 等探索性工作放在独立 sandbox，不影响生产主线。

1.2 分层架构图



1.3 核心数据流



2. 产品功能规格

2.1 产品定位

Hermass QuantMind 是一个 Agent-Native 的 AI 量化策略平台，面向个人投资者和小型量化团队，提供"自然语言策略生成 → 智能回测 → 多 Agent 协作决策 → 模拟执行 → 持续进化"的全闭环能力。

2.2 目标用户

角色	画像	核心诉求
进阶散户	有 3-5 年交易经验，了解基本面和技术面	系统化自己的策略思路，降低情绪干扰
小型私募研究员	有编程基础但不想从零搭建系统	快速验证策略想法，多策略组合管理
量化爱好者	学过金融工程，想练手实盘模拟	低门槛的策略实验环境

2.3 自然语言策略生成 (Strategy Lab)

功能	描述	优先级	来源
中文意图解析	"MA5上穿MA20买入，跌破MA10卖出" → LLM 解析	P0	Qoder
策略 DSL v2 生成	解析结果 → 标准 JSON Schema (含 hypothesis/risk/evaluation)	P0	Qoder + Codex
条件块预览	展示每个入场/出场/过滤条件的命中范围	P0	Qoder
策略模板库	内置 VCP/MA2560/Bollinger/ATR 吊灯等模板	P0	Qoder
参数编辑器	可视化编辑 DSL 参数 (MA 周期、止损幅度等)	P1	Qoder
策略组合编排	多策略权重配置 + 联合入场	P2	Qoder
版本管理	策略版本树 + diff 对比	P1	Qoder
用户可读解释	同步生成"机器 DSL + 人类解释"双份输出	P0	Codex
追问澄清	语义不完整时先追问，不瞎补	P0	Codex

约束： - 不允许用户直接提交 Python 代码作为策略。 - DSL 翻译层必须是确定性的（纯函数，零 LLM 依赖）。 - 所有 DSL 必须通过 JSON Schema + Pydantic 校验后才能执行。

2.4 智能策略回测

功能	描述	优先级	来源
一键回测	DSL → 回测引擎 → 结果 (< 30s Light / 后台 Full)	P0	Qoder + Codex
Walk-Forward	至少 3 折滚动验证 + 样本外评价	P0	Qoder
绩效指标	年化收益/最大回撤/夏普/胜率/盈亏比	P0	Qoder
环境分层	按 MN1 State 分层展示不同市场阶段的表现	P0	Qoder
基准对比	策略 vs 沪深300 vs 全市场等权	P1	Qoder
交易明细	每笔交易记录含入场原因	P0	Qoder
成本模型	佣金(万三) + 印花税(千五) + 滑点(千一)	P0	Qoder
报告导出	Markdown + HTML tearsheet	P1	Qoder
历史对比	同策略不同版本回测对比	P2	Qoder
Polars 热路径	核心计算路径用 Polars 向量化加速	P1	Kimi

性能要求： - 单策略单品种 252 天回测 < 5s (Light 模式 < 30s) 。 - 全市场 5000+ 品种一年回测 < 30s (Light) 。 - 支持增量回测（只计算新增交易日）。

2.5 多 Agent 协作系统

Agent	职责	可改什么	不可改什么	来源
Strategy Designer	中文→DSL 翻译、策略草稿	生成候选策略	不能修改已批准策略	Qoder
Backtest Critic	解读回测结果、识别过拟合	发出改进建议	不能标记为"必胜"	Qoder
Risk Guardian	检查红线、仓位、极端情况	风险评级、否决	不能放宽红线	Qoder
Contraction Observer	监控波幅收缩、VCP 形态	发出收缩信号	不能代替入场	Qoder
Trend Analyst	W1/D1 趋势分析、MA 结构	趋势评估	不能单独拍板	Qoder
Market Macro	宏观四维（流动性/增长/估值/情绪）	宏观评分	不能覆盖微观信号	Qoder
Industry Chain	**产业链上下游传导分析**	**产业评分**	**不能替代技术信号**	**Kimi（新增）**
Execution Coach	生成模拟执行计划	Paper Order	不能真实下单	Qoder
Review Agent	每日复盘、策略进化	评价和提醒	不能自动升格策略	Qoder
Dynamic Weight Router	**综合 8 Agent 意见分配权重**	**权重分配**	**不能修改策略本身**	**Kimi（新增）**

交互模式： 1. 辩论制：Agent 间意见冲突时展示分歧点，由 Router 加权。 2. 常驻反驳：Risk Guardian 永远作为反方。 3. 人类最终确认：所有执行级决策需人类点击确认。

2.6 实时市场分析

功能	描述	优先级	来源
多周期状态看板	MN1/W1/D1 State 矩阵 + EF 宽度	P0	Qoder

功能	描述	优先级	来源
行业轮动	25 行业 EF 分布 + 近期信号	P0	Qoder
个股 State Card	三周期共振度 + 策略适配评分	P0	Qoder
宏观象限	四维宏观评分 + 当前象限判定	P1	Qoder
产业链观察池	上下游关联品种状态聚合	P1	Kimi
异常告警	市场极端情况主动推送	P1	Qoder
资金流跟踪	行业主力资金流向	P1	Qoder

2.7 前沿研究 Sandbox（不影响主线）

方向	描述	优先级	来源
TS-FM 预测层	Chronos ONNX 接入 State Cube，验证 A 股预测效果	研究	Kimi
RAG-KG Agent	Kuzu 知识图谱，改造 Agent 加入检索能力	研究	Kimi
策略进化账本	AgentMemory 记录每次判断 → 后验反馈 → 权重调整	P2	Kimi
Obsidian 知识沉淀	对话/文档/复盘自动同步本地知识库	P2	Kimi

3. 技术架构详细设计

3.1 Strategy DSL Engine

位置：hermass_platform/strategy_lab/

3.1.1 DSL v2 Schema

融合 Qoder 的条件结构 + Codex 的 hypothesis/risk/evaluation/provenance：

```
{ "$schema": "https://json-schema.org/draft/2020-12/schema", "type": "object", "required": ["strategy_id", "name", "schema_version", "entry", "exit", "risk"], "properties": {
  "strategy_id": {"type": "string", "pattern": "^[a-z0-9_]+$"}, "name": {"type": "string", "maxLength": 64}, "schema_version": {"const": "strategy_dsl_v2"}, "description":
{"type": "string"}, "hypothesis": { "type": "object", "properties": { "summary": {"type": "string"}, "market_regime": {"type": "array", "items": {"type": "string"}} } },
"entry": { "type": "array", "items": {"$ref": "#/$defs/condition_block"}, "minItems": 1 }, "filters": { "type": "array", "items": {"$ref": "#/$defs/condition_block"} }, "exit":
{ "type": "array", "items": {"$ref": "#/$defs/condition_block"}, "minItems": 1 }, "risk": { "type": "object", "properties": { "risk_per_trade": {"type": "number"},
"max_position_pct": {"type": "number", "maximum": 0.25}, "stop_loss_required": {"type": "boolean", "const": true} } }, "evaluation": { "type": "object", "properties": {
"walk_forward_required": {"type": "boolean"}, "min_oos_trades": {"type": "integer"} } }, "execution": { "type": "object", "properties": { "mode": {"type": "string", "enum":
["paper"]}, "human_confirm_required": {"type": "boolean", "const": true} } }, "provenance": { "type": "object", "properties": { "created_by": {"type": "string"},
"source_message_id": {"type": "string"} } }, "metadata": { "type": "object", "properties": { "author": {"type": "string"}, "tags": {"type": "array", "items": {"type":
"string"}}, "suitable_environments": {"type": "array", "items": {"type": "string"}} } } } }
```

3.1.2 条件类型注册表

技术	应用场景	预期提升	来源
DuckDB 窗口函数	指标计算（MA/BB/ATR）	5-20x	Qoder
Polars 向量化	信号生成、权益计算热路径	10-50x	Kimi
预计算缓存	MA/BB/ATR 预存在 Foundation DB	即时命中	Qoder
增量回测	只重算新增交易日	大幅减少	Qoder
并行 Walk-Forward	多折并行执行	线性扩展	Qoder

3.3 Agent Orchestration（多 Agent 编排）

位置：agently_adapter/

3.3.1 Agent Debate DAG

```
strategy_debate_dag: trigger: backtest_complete | user_request nodes: █ load_context # █ State Cube + █ █ critic_review # Backtest Critic █ █ risk_review # Risk Guardian █ █ trend_assessment # Trend Analyst █ █ contraction_check # Contraction Observer █ █ macro_assessment # Market Macro █ █ industry_chain_check # Industry Chain █ █ ← Kimi █ weight_router # Dynamic Weight Router █ ← Kimi █ synthesize_opinion # █ █ save_judgment # █ AgentMemory + Trace ← Codex
```

3.3.2 Agent 输出契约（Typed JSON）

```
{ "agent_id": "risk_guardian", "verdict": "caution", "confidence": 0.78, "reasoning": "██████ 18% ██████ 15%██████ 5 █...", "suggestions": [ {"type": "reduce_position", "detail": "█ max_position_pct █ 10% █ 7%"}, {"type": "add_filter", "detail": "████ ADX>25 ██████"} ], "conflicts_with": ["trend_analyst"], "resonances_with": ["contraction_observer"], "data_sources": ["backtest_result", "state_cube"], "trace_span_id": "span_abc123" // ← Codex }
```

3.3.3 Router 权重机制（Kimi）

```
# Dynamic Weight Router ██████ weight_factors = { "agent_historical_accuracy": 0.30, # Agent ██████ "market_phase_relevance": 0.25, # ██████ Agent ██████ "data_freshness": 0.20, # ██████ "conflict_resolution": 0.15, # ██████████ "user_preference": 0.10, # ██████ }
```

3.4 MCP Tool Registry（Codex 融合）

位置：hermass_platform/tools/registry.py

```
# MCP ████████████████████ MCP ████ def register_tool(name: str, schema: dict, handler: callable, tags: list[str]): """████████ Agent ██████""" def list_tools(policy: str = "default") -> list[dict]: """██████████████████""" def invoke_tool(tool_name: str, args: dict, context: dict) -> dict: """██████████████████"""
```

优先纳入的工具：

工具	功能	权限
State Cube query	查询多周期状态	全部 Agent
Foundation query	查询 K 线/指标	全部 Agent
Backtest preview	快速预览命中	Strategy Designer
Walk-forward runner	滚动验证	Backtest Critic

工具	功能	权限
Industry chain lookup	产业链关联查询	Industry Chain
Macro prior lookup	宏观先验查询	Market Macro
Paper order planner	模拟订单规划	Execution Coach

3.5 Observability (Codex 融合)

位置：hermass_platform/observability/

模块	职责	兼容目标
`tracing.py`	Agent trace / Tool call trace / LLM call trace	Phoenix / OpenInference
`evals.py`	策略建议接受率 / Agent 准确率 / 回测质量评分	Phoenix Datasets
`exporters.py`	OpenTelemetry 格式导出	OTLP

Trace 记录内容：- 每个 Agent 节点的输入/输出/延迟 - Tool 调用的参数/结果/失败率 - LLM 调用的 token 数/延迟/成本 - 回测建议是否被用户接受

4. 数据库设计

4.1 StrategyLab.duckdb (新增)

路径：outputs/strategy_lab/StrategyLab.duckdb

```
-- ■■■■■■ CREATE TABLE user_strategies ( strategy_id VARCHAR PRIMARY KEY, user_id VARCHAR NOT NULL DEFAULT 'default', name VARCHAR NOT NULL, description VARCHAR, status VARCHAR NOT NULL DEFAULT 'draft', -- draft|active|archived|rejected current_version_id VARCHAR, template_source VARCHAR, tags VARCHAR[], created_at TIMESTAMP DEFAULT current_timestamp, updated_at TIMESTAMP DEFAULT current_timestamp ); -- ■■■■■■ CREATE TABLE strategy_versions ( version_id VARCHAR PRIMARY KEY, strategy_id VARCHAR NOT NULL REFERENCES user_strategies(strategy_id), parent_version_id VARCHAR, version_number INTEGER NOT NULL, dsl_json JSON NOT NULL, change_summary VARCHAR, created_by VARCHAR NOT NULL, -- 'user' | 'agent:xxx' created_at TIMESTAMP DEFAULT current_timestamp ); CREATE INDEX idx_sv_strategy ON strategy_versions(strategy_id); -- ■■■■■■ CREATE TABLE strategy_backtests ( backtest_id VARCHAR PRIMARY KEY, strategy_id VARCHAR NOT NULL, version_id VARCHAR NOT NULL, start_date DATE NOT NULL, end_date DATE NOT NULL, initial_capital DOUBLE NOT NULL, data_version VARCHAR, total_return DOUBLE, annual_return DOUBLE, max_drawdown DOUBLE, sharpe_ratio DOUBLE, win_rate DOUBLE, profit_factor DOUBLE, trade_count INTEGER, metrics_json JSON, state_breakdown_json JSON, walk_forward_json JSON, risk_flags VARCHAR[], report_path VARCHAR, trades_path VARCHAR, status VARCHAR NOT NULL DEFAULT 'running', -- running|completed|failed elapsed_seconds DOUBLE, trace_id VARCHAR, -- ← Codex created_at TIMESTAMP DEFAULT current_timestamp ); CREATE INDEX idx_bt_strategy ON strategy_backtests(strategy_id, version_id); -- AI ■■■■■■ CREATE TABLE strategy_improvements ( proposal_id VARCHAR PRIMARY KEY, strategy_id VARCHAR NOT NULL, source_backtest_id VARCHAR, proposal_type VARCHAR NOT NULL, proposal_dsl_diff JSON NOT NULL, agent_id VARCHAR NOT NULL, risk_review_json JSON, accepted BOOLEAN DEFAULT NULL, result_backtest_id VARCHAR, created_at TIMESTAMP DEFAULT current_timestamp ); -- ■■■■■■ CREATE TABLE paper_orders ( order_id VARCHAR PRIMARY KEY, strategy_id VARCHAR NOT NULL, version_id VARCHAR, stock_code VARCHAR NOT NULL, signal_date DATE NOT NULL, side VARCHAR NOT NULL, planned_price DOUBLE NOT NULL, planned_size INTEGER NOT NULL, actual_price DOUBLE, actual_size INTEGER, status VARCHAR NOT NULL DEFAULT 'pending', red_line_check_json JSON, human_approved BOOLEAN, reject_reason VARCHAR, created_at TIMESTAMP DEFAULT current_timestamp, executed_at TIMESTAMP ); CREATE INDEX idx_po_date ON paper_orders(signal_date); -- ■■■■■■ CREATE TABLE paper_positions ( position_id VARCHAR PRIMARY KEY, strategy_id VARCHAR NOT NULL, stock_code VARCHAR NOT NULL, entry_date DATE NOT NULL, entry_price DOUBLE NOT NULL, shares INTEGER NOT NULL, current_price DOUBLE, stop_price DOUBLE, target_price DOUBLE, unrealized_pnl DOUBLE, status VARCHAR NOT NULL DEFAULT 'open', exit_date DATE, exit_price DOUBLE, exit_reason VARCHAR, created_at TIMESTAMP DEFAULT current_timestamp ); CREATE INDEX idx_pp_open ON paper_positions(status, strategy_id); -- ■■■■■■ CREATE TABLE strategy_audit_log ( log_id VARCHAR PRIMARY KEY, timestamp TIMESTAMP DEFAULT current_timestamp, action VARCHAR NOT NULL, strategy_id VARCHAR, actor VARCHAR NOT NULL, detail_json JSON, red_line_triggered BOOLEAN DEFAULT FALSE ); CREATE INDEX idx_audit_time ON strategy_audit_log(timestamp);
```

4.2 AgentMemory 扩展

在现有 outputs/agent_memory/AgentMemory.duckdb 新增：

```
-- Agent  CREATE TABLE strategy_agent_judgments ( judgment_id VARCHAR PRIMARY KEY, strategy_id VARCHAR NOT NULL, backtest_id VARCHAR, agent_id VARCHAR NOT NULL, verdict VARCHAR NOT NULL, confidence DOUBLE, reasoning TEXT, suggestions_json JSON, conflicts_json JSON, resonances_json JSON, trace_span_id VARCHAR, -- ← Codex created_at TIMESTAMP DEFAULT current_timestamp ); -- outcome← Kimi CREATE TABLE strategy_outcomes ( outcome_id VARCHAR PRIMARY KEY, strategy_id VARCHAR NOT NULL, version_id VARCHAR NOT NULL, evaluation_date DATE NOT NULL, forward_return_5d DOUBLE, forward_return_20d DOUBLE, hit_count INTEGER, miss_count INTEGER, notes VARCHAR, created_at TIMESTAMP DEFAULT current_timestamp ); -- Agent trace  CREATE TABLE agent_traces ( trace_id VARCHAR PRIMARY KEY, session_id VARCHAR, node_name VARCHAR, status VARCHAR, latency_ms INTEGER, input_json VARCHAR, output_json VARCHAR, created_at TIMESTAMP DEFAULT current_timestamp ); -- Tool call  CREATE TABLE tool_calls ( call_id VARCHAR PRIMARY KEY, session_id VARCHAR, tool_name VARCHAR, args_json VARCHAR, result_json VARCHAR, status VARCHAR, latency_ms INTEGER, created_at TIMESTAMP DEFAULT current_timestamp );
```

5. Agent 体系设计

5.1 Agent 全景图

层级	Agent	核心职责	输出	权限
分析师层	Trend Analyst	W1/D1 趋势、MA 结构评估	趋势评分 + 方向	只读
	Momentum Agent	动量强度、量价配合	动量评分	只读
	Volatility Agent	波动率状态、带宽位置	波动评估	只读
	Boundary Agent	支撑阻力、边界状态	边界位置	只读
	Industry Chain	**产业链上下游传导**	**产业评分 + 关联信号**	**只读**
辩论层	Bull Researcher	多头论据收集	结构化证据	只读
	Bear Researcher	空头论据收集	结构化证据	只读
决策层	Strategy Designer	中文→DSL 翻译	DSL JSON	写草稿
	Backtest Critic	回测结果解读、过拟合识别	评审意见	写建议
	Optimizer	参数/条件改进建议	Proposal	写建议
风控层	Risk Guardian	红线检查、风险评级	通过/警告/否决	**一票否决**
路由层	**Dynamic Weight Router**	**综合加权、冲突识别**	**权重分配 + 结论**	**只读**
执行层	Execution Coach	模拟执行计划	Paper Order	写计划
	Review Agent	每日复盘、策略退化监控	复盘报告	写报告

5.2 Agent 消息总线 (AgentBus)

延续现有 hermass_platform/bus/agent_bus.py 的 6 类标准消息，新增策略生命周期 topic：

Topic	触发时机	消费方
`strategy_created`	新策略生成	Review Agent

Topic	触发时机	消费方
`backtest_completed`	回测完成	Critic / Risk / Router
`debate_concluded`	辩论结束	Execution Coach
`paper_order_proposed`	模拟订单生成	Risk Guardian
`strategy_degraded`	策略表现退化	Review Agent / Human
`review_needed`	需人工复核	Human Reviewer

6. API 接口规范

6.1 Strategy Lab API

端点	方法	说明	来源
`/api/strategy-lab/generate`	POST	中文 → DSL v2 生成	Qoder
`/api/strategy-lab/validate`	POST	校验 DSL 合法性	Qoder
`/api/strategy-lab/preview`	POST	条件命中预览	Qoder
`/api/strategy-lab/backtest`	POST	启动 Light/Full 回测	Qoder
`/api/strategy-lab/backtest/{id}`	GET	查询回测结果	Qoder
`/api/strategy-lab/improve`	POST	AI 生成改进建议	Qoder
`/api/strategy-lab/library`	GET	策略列表	Codex
`/api/strategy-lab/proposals`	GET	Proposal 列表	Codex

6.2 Agent Debate API

端点	方法	说明
`/api/debate/run`	POST	运行多 Agent 辩论
`/api/debate/{debate_id}`	GET	查询辩论结果

6.3 Paper Trading API

端点	方法	说明
`/api/paper/orders`	POST	创建模拟订单
`/api/paper/orders/{id}`	GET	查询订单状态
`/api/paper/positions`	GET	查询持仓

6.4 Observability API (Codex 融合)

端点	方法	说明
`/api/agent-traces/{session_id}`	GET	Agent trace 查询
`/api/agent-evals/{strategy_id}`	GET	策略评审统计
`/api/tool-usage/{session_id}`	GET	Tool call trace

7. 安全、合规与风控

7.1 系统级安全

层级	机制	实现
输入安全	DSL Schema + Pydantic 校验	JSON Schema + Python 校验
代码安全	**禁止执行 LLM 生成代码**	DSL → SQL/ Polars 纯函数翻译
权限控制	用户身份隔离	user_id 绑定 + API Token
审计追踪	全操作日志	strategy_audit_log + agent_traces
速率限制	API 频率控制	10 req/min (回测) , 60 req/min (查询)
工具安全	MCP Tool Policy	Agent 只能访问白名单工具

7.2 策略级风控 (Red Lines 集成)

```
def check_strategy_redlines(dsl: dict) -> RedLineResult: """ 1. max_position_pct (25%) 2. 3. 4. kill-switch 5. < 48 """
def check_execution_redlines(order: PaperOrder) -> RedLineResult: """ 1. 10% 2. 30% 3. 4. >48h 5. """
```

7.3 合规体系 (融合 DeepSeek)

维度	实现	级别
投资者适当性	风险测评问卷 (10 题) → C1-C5 分级	P1 (M3 前)
风险提示	全站免责声明 + 每页底部可见	P0
强制弹窗	首次登录风险确认 (M3 参考)	P1
禁止事项	不承诺收益、不推荐具体时点、不代客理财	P0
审计日志	全操作 WORM 存储, 保留 5 年	P1
数据加密	TLS 1.3 传输 (部署层)	P1

7.4 风控 Agent 独立审查

Risk Guardian Agent 拥有"一票否决权"：

• 回测最大回撤超阈值 → risk_flag: excessive_drawdown - 样本外表现显著差于样本内 → risk_flag: overfitting - 连续亏损超过设定次数 → risk_flag: losing_streak - 策略集中度过高 → risk_flag: concentration - 数据过期 > 48h → risk_flag: stale_data

8. 可观测性设计

8.1 Trace Schema (Codex 融合)

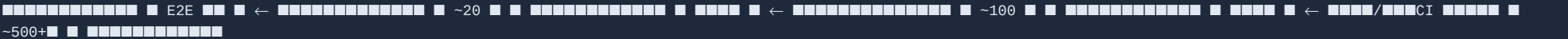
```
{ "trace_id": "trace_xxx", "session_id": "session_xxx", "strategy_id": "strategy_xxx", "nodes": [ { "node_name": "risk_guardian", "status": "completed", "latency_ms": 450, "input_hash": "sha256:abc...", "output_hash": "sha256:def...", "tool_calls": [ {"tool": "backtest_report", "latency_ms": 120} ] }, {"node_name": "backtest_engine", "status": "completed", "latency_ms": 2800}], "total_latency_ms": 3200, "created_at": "2026-06-06T10:00:00Z" }
```

8.2 Eval 指标

指标	定义	目标
DSL 合法性	中文输入 → 合法 DSL 比例	> 85%
Agent 准确率	判断与后续 outcome 一致率	> 60%
建议接受率	用户接受 Agent 改进建议比例	> 30%
回测完成率	回测任务不超时不报错	> 95%
Trace 完整率	关键路径 trace 落库比例	> 99%

9. 测试规范 (融合 DeepSeek)

9.1 测试金字塔



9.2 各模块测试标准

模块	覆盖率	重点测试
DSL 解析器	≥ 95%	条件类型全覆盖、边界值、非法输入
回测引擎	≥ 90%	绩效指标精度、临界日期、Polars/DuckDB 一致性
沙箱编译器	≥ 95%	DAG 环检测、条件死代码消除
Agent 输出	≥ 85%	JSON Schema 校验、冲突检测

模块	覆盖率	重点测试
红线检查	≥ 95%	所有红线场景触发验证

9.3 压力测试

场景	目标	工具
API 并发查询	500 QPS，P95 < 200ms	k6 / Locust
并发回测任务	20 任务同时，无超时	自定义脚本
WebSocket 连接	500 连接同时，延迟 < 100ms	Artillery
LLM 推理并发	20 并发，P95 < 5s	k6

10. 实施路线图

10.1 总体时间线



10.2 分阶段详细计划

Phase 0：底座整理（2 周）

任务	路径	产出
DSL v2 Schema 定义	`hermass_platform/strategy_lab/dsl_schema.py`	JSON Schema + Pydantic Model
条件注册表	`hermass_platform/strategy_lab/condition_registry.py`	12+ 条件类型
翻译器扩展	`hermass_platform/strategy/condition_translator.py`	完整翻译
DSL 校验器	`hermass_platform/strategy_lab/dsl_validator.py`	语义+结构校验
LLM 生成器	`hermass_platform/strategy_lab/dsl_generator.py`	中文→DSL
StrategyLab DB	`outputs/strategy_lab/StrategyLab.duckdb`	DDL 迁移
AgentMemory 扩展	DDL	strategy_agent_judgments / outcomes / traces

验收：- 输入"MA5上穿MA20买入"→ 生成合法 DSL v2 - DSL 通过 Schema + Pydantic 校验 - .venv/bin/python 编译全项目通过

Phase 1：Strategy Lab MVP（4 周）

任务	路径	产出
API: generate	`web/main.py`	POST /api/strategy-lab/generate
API: validate	`web/main.py`	POST /api/strategy-lab/validate
API: preview	`web/main.py`	POST /api/strategy-lab/preview
前端编辑器	`web/templates/strategy_lab.html`	条件块可视化
策略模板库	`config/strategy_templates/`	5+ 内置模板

验收：- 10 组中文输入 ≥ 8 组生成合法 DSL - Preview 返回命中结果 - 非法 DSL（如缺止损）被拒绝并提示原因

Phase 2：回测引擎（4 周）

任务	路径	产出
DSL→回测适配器	`backtest/dsl_runner.py`	DSL 入口函数
Polars 热路径	`backtest/engine.py`	关键计算向量化
Walk-Forward	`backtest/walk_forward.py`	3 折验证
异步回测 API	`web/main.py`	POST + GET 状态查询
报告生成	`backtest/report.py`	HTML tearsheet
State 分层	`backtest/metrics.py`	env_breakdown

验收：- Light Backtest P95 < 30s - Walk-Forward ≥ 3 折 - 报告含完整指标 + 交易明细 + 分层统计

Phase 3：Agent Debate（4 周）

任务	路径	产出
Debate DAG	`agently_adapter/scenarios/strategy_debate.py`	DAG 定义
Critic Agent	`agently_adapter/agents/critic.py`	回测评审
Industry Chain Agent	`hermass_platform/agents/industry_chain.py`	产业链分析
Weight Router	`agently_adapter/agents/weight_router.py`	动态权重
Debate API	`web/main.py`	`/api/debate/run`
红线集成	`hermass_platform/red_lines.py`	策略级检查

验收：- 回测完成后 ≥ 3 个 Agent 输出意见 - Router 正确识别冲突与共振 - 仓位超限 → 拒绝并审计

Phase 4：协议与可观测性（4 周）

任务	路径	产出
Tool Registry	`hermass_platform/tools/registry.py`	MCP 风格注册

任务	路径	产出
Tracing 落库	`hermass_platform/observability/tracing.py`	Agent/Tool trace
Eval 框架	`hermass_platform/observability/evals.py`	准确率统计
Pydantic Contracts	`hermass_platform/ai_contracts/`	Typed Agent 输出
Trace API	`web/main.py`	`/api/agent-traces/*`

验收：- ≥ 5 个 Tool 可注册调用 - 每次 Agent Review 有 trace 落库 - 策略建议接受率可统计

Phase 5：Paper Trading + 复盘（4 周）

任务	路径	产出
订单管理	`hermass_platform/paper_trading/order_manager.py`	CRUD
持仓跟踪	`hermass_platform/paper_trading/position_tracker.py`	实时 PnL
执行账本	`hermass_platform/paper_trading/execution_ledger.py`	不可篡改
每日对账	`hermass_platform/paper_trading/daily_reconcile.py`	自动化
Review Agent	`agently_adapter/agents/review.py`	每日复盘
复盘→Obsidian	`tools/obsidian_exporter/`	知识沉淀

验收：- 模拟订单需人类确认 - 红线拦截生效 - 每日自动对账 + 复盘

Phase 6：前沿探索（持续）

方向	路径	目标
TS-FM 接入	`hermass_platform/research/tsfm_sandbox/`	Chronos ONNX 验证 A 股效果
RAG-KG	`hermass_platform/research/rag_kg_sandbox/`	Kuzu 知识图谱可行性
A2A 适配	`hermass_platform/protocols/a2a_adapter.py`	预留外部 Agent 接入

11. 验收标准

11.1 功能验收

#	验收项	判定标准	阶段
1	中文输入生成合法 DSL	10 组测试用例 ≥ 8 组通过	Phase 1
2	非法 DSL 被拒绝	输入缺少止损 → 校验失败并给出原因	Phase 1
3	回测 < 30s (Light)	全市场 252 天，P95 < 30s	Phase 2

#	验收项	判定标准	阶段
4	Walk-Forward ≥ 3 折	输出含 3+ 折指标	Phase 2
5	Agent 输出结构化	JSON Schema 校验通过	Phase 3
6	冲突/共振识别	Router 正确标记冲突 Agent	Phase 3
7	风控红线拦截	仓位超限 → 拒绝并审计	Phase 3
8	MCP Tool 注册	≥ 5 个工具可注册调用	Phase 4
9	Trace 可落库	每次 review 有 node trace	Phase 4
10	Paper Order 人类确认	未确认不执行	Phase 5
11	每日自动复盘	盘后生成复盘摘要	Phase 5
12	审计日志完整	每个关键操作有 log	全阶段
13	py_compile 通过	`.venv/bin/python` 无语法错误	全阶段
14	免责声明可见	每个页面底部	Phase 1
15	数据过期拦截	>48h 数据 → 拒绝执行建议	全阶段

11.2 性能验收

指标	目标	验证方式
Light Backtest P95	< 30s	k6 压测 20 并发
API 查询 P95	< 2s	k6 压测 500 QPS
首屏加载	< 3s	Lighthouse
并发回测	20 任务无超时	自定义脚本

12. 风险矩阵与缓解

风险	影响	概率	缓解措施
LLM 解析歧义	生成策略不符预期	高	条件块确认 + 置信度 + 追问澄清 (Codex)
回测超时	用户体验差	中	Light/Full 分层 + Polars 加速 (Kimi) + 异步
过拟合	策略失效	高	Walk-Forward + Critic Agent + 样本外标记
Agent 幻觉	错误建议	中	Typed Contract (Codex) + 数据源标注 + 人类确认
数据过期	决策基于旧数据	中	data_freshness gate + >48h 拒绝
安全漏洞	注入攻击	低	DSL 白名单 + SQL 参数化 + **不执行代码**
前沿框架变更	兼容层失稳	中	Adapter 模式接入，不深绑 (Codex)

风险	影响	概率	缓解措施
合规风险	法律/资金风险	中	Paper-only + 人类确认 + C1-C5 分级 (DeepSeek)

13. 成本估算（融合 DeepSeek 方法）

13.1 MVP 阶段（本地部署，0-6 个月）

资源	规格	月费（人民币）
开发服务器	本机 Mac Studio	0（已有）
DeepSeek API	按调用量	~500-1,000
黑狼数据 API	按调用量	~300-500
合计		**~800-1,500/月**

13.2 V1.0 阶段（服务器扩展，6-12 个月）

资源	规格	月费（人民币）
云服务器（2台）	8C16G	~1,000
GPU 推理（可选）	A10 24GB	~3,000-5,000
对象存储	500GB	~100
CDN / 带宽	50Mbps	~500
合计（无 GPU）		**~1,600/月**
合计（含 GPU）		**~4,600-6,600/月**

14. 附录

14.1 参考文档清单

模型	原始 PRD	原始技术文档	版本
Qoder（基础）	`AI_QUANT_PLATFORM_PRD_QODER_更新版.md`	`AI_QUANT_PLATFORM_ARCHITECTURE_QODER_更新版.md`	v2.0
Kimi（融合）	`PRD_AI_QUANT_STUDIO_KIMI.md`	`TRD_AI_QUANT_STUDIO_KIMI.md`	v2.0
Codex（融合）	`CODEX_AI_QUANT_PLATFORM_PRD_更新版.md`	`CODEX_AI_QUANT_PLATFORM_TECHNICAL_DESIGN_更新版.md`	v2.0

模型	原始 PRD	原始技术文档	版本
DeepSeek（融合）	`AI_QUANT_PLATFORM_PRD_deepseek.md`	`AI_QUANT_PLATFORM_TECHNICAL_DESIGN_deepseek.md`	v1.0

14.2 术语表

术语	说明
DSL	Domain Specific Language，策略领域特定语言
MCP	Model Context Protocol，模型上下文协议
A2A	Agent-to-Agent Protocol，智能体间互操作协议
TS-FM	Time Series Foundation Model，时序基础模型
RAG-KG	Retrieval-Augmented Generation + Knowledge Graph
Walk-forward	滚动窗口样本外验证方法
State Cube	多周期多指标状态全景数据
WORM	Write Once Read Many，一次写入多次读取

14.3 文档历史

版本	日期	变更说明
v3.0 Final	2026-06-06	基于 Qoder 架构，融合 Kimi/Codex/DeepSeek 精华，形成可执行最终方案

> 本文档性质：最终版实施方案（Executable Plan）> 适用范围：Hermass AI Quant Platform Phase 0 ~ Phase 6> 维护责任：各阶段负责人需在里程碑完成时更新本章节的验收状态